实验报告

课程名称：Design pattern and software architecture

学　　院：计算机科学与工程学院

专　　业:Computer Science and Technology　班　　级：SE-9

姓　　名：Muhammad Samudera Bagja　学　　号：

年　　月　　日

山东科技大学教务处制

实 验 报 告

组　别	1	姓　名	Samudera	同组实验者	None
实验项目名称	Strategy Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:		指导教师（签名）： 2026 3 23			

1. Objective

The objective of this experiment is to implement the Strategy design pattern using an E-Commerce Payment Gateway example involving a Context (`PaymentProcessor`) and various payment strategies (`MidtransVAStrategy`, `StripeCreditCardStrategy`, `XenditEWalletStrategy`, `CryptoWalletStrategy`). The goal of this pattern is to define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

In this example, the program has the following structure:

- **Context** (`PaymentProcessor`): maintains a reference to a `PaymentStrategy` object and delegates the payment execution to it. It does not implement the payment algorithm itself.
- **Strategy Interface** (`PaymentStrategy`): defines a common interface for all supported algorithms. The context uses this interface to call the algorithm defined by a Concrete Strategy.
- **Concrete Strategies** (`MidtransVAStrategy`, `StripeCreditCardStrategy`, `XenditEWalletStrategy`, `CryptoWalletStrategy`): implement the algorithm using the `PaymentStrategy` interface. Each represents a different payment method.
- **Data Transfer Objects** (`PaymentRequest`, `PaymentResponse`): carry the input data for the payment and return the status of the transaction.
- **Client/Controller** (`PaymentController`): acts as the client that determines which specific strategy to pass to the context based on the user's selected payment method.

2. Principle

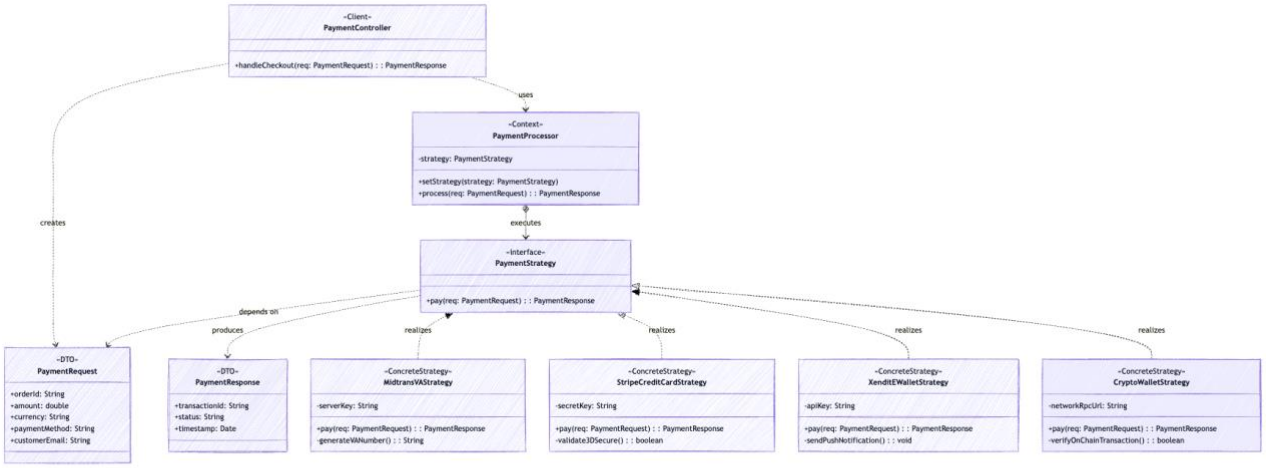
The Strategy pattern defines a set of algorithms and encapsulates them so they can be selected at runtime.

Instead of having a single class with complex conditional statements (like `if-else` or `switch`) to choose between different behaviors, the Strategy pattern extracts these behaviors into separate, interchangeable classes. The Context object (`PaymentProcessor`) delegates the work to a linked strategy object instead of executing it on its own. The Context only interacts with the strategies via the generic `PaymentStrategy` interface, which declares the `pay(PaymentRequest)` method. The `PaymentController` is responsible for instantiating the appropriate strategy based on the `paymentMethod` string and setting it on the Context.

Key benefits of this approach include:

- **Open/Closed Principle:** You can introduce new payment strategies (e.g., `PayPalStrategy`) without having to change the `PaymentProcessor` context or the other existing strategies.
- **Isolation:** The implementation details, data, and dependencies of specific algorithms are hidden from the context.
- **Avoidance of conditionals:** Replaces massive conditionals with object composition and delegation, making the code much easier to read and maintain.

3. UML Diagram



4. Key Code (Java)

```
// Data Transfer Object (immutable)

static final class VpnHealthContext {

    private final String nodeId;

    private final String status;

    private final int pingMs;

    private final LocalDateTime timestamp;

    private final boolean isDown;



    public VpnHealthContext(String nodeId, String status, int pingMs,

                            LocalDateTime timestamp, boolean isDown) {

        this.nodeId    = nodeId;

        this.status    = status;

        this.pingMs    = pingMs;

        this.timestamp = timestamp;

// Strategy Interface

interface PaymentStrategy {

    PaymentResponse pay(PaymentRequest req);

}



// Concrete Strategy 1

class MidtransVAStrategy implements PaymentStrategy {
```

```
    private String serverKey;
    public MidtransVAStrategy(String serverKey) { this.serverKey = serverKey; }

    @Override
    public PaymentResponse pay(PaymentRequest req) {
        System.out.println("[Midtrans] Processing payment via Virtual Account...");
        String vaNumber = generateVANumber();
        System.out.println("[Midtrans] VA Number for " + req.customerEmail + " : " + vaNumber);
        return new PaymentResponse(UUID.randomUUID().toString(), "PENDING_PAYMENT");
    }
    private String generateVANumber() { return "8077" + (int)(Math.random() * 10000000); }
}

// Concrete Strategy 2
class StripeCreditCardStrategy implements PaymentStrategy {
    private String secretKey;
    public StripeCreditCardStrategy(String secretKey) { this.secretKey = secretKey; }

    @Override
    public PaymentResponse pay(PaymentRequest req) {
        System.out.println("[Stripe] Processing Credit Card payment of " + req.amount + " " + req.currency);
        if (validate3DSecure()) {
```

```
                System.out.println("[Stripe] 3D Secure Valid! Payment Successful.");
                return new PaymentResponse(UUID.randomUUID().toString(), "SUCCESS");
            } else {
                System.out.println("[Stripe] 3D Secure Failed.");
                return new PaymentResponse(UUID.randomUUID().toString(), "FAILED");
            }
        }
        private boolean validate3DSecure() { return true; }
}

// Context Class
class PaymentProcessor {
    private PaymentStrategy strategy;

    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public PaymentResponse process(PaymentRequest req) {
        if (this.strategy == null) {
            throw new IllegalStateException("Payment Strategy has not been set!");
        }
```

```
        return this.strategy.pay(req);
    }
}

// Client / Controller Layer
class PaymentController {
    public PaymentResponse handleCheckout(PaymentRequest req) {
        PaymentProcessor processor = new PaymentProcessor();

        // Select payment strategy based on request input
        switch (req.paymentMethod.toUpperCase()) {
            case "MIDTRANS_VA":
                processor.setStrategy(new MidtransVAStrategy("server-key-md-123"));
                break;
            case "STRIPE_CC":
                processor.setStrategy(new StripeCreditCardStrategy("sk_test_4eC39Hq"));
                break;
            // Other cases omitted for brevity
            default:
                throw new IllegalArgumentException("Payment method not supported: " + req.paymentMethod);
        }
        // Execute payment via Context
```

```
        return processor.process(req);

    }

}
```

5. Analysis

Advantages: The Strategy pattern significantly cleans up code that would otherwise be littered with massive conditional statements. In this program, `PaymentProcessor` acts as the context and only requires a valid `PaymentStrategy` object to execute the payment; it is completely decoupled from the actual logic of processing credit cards or verifying blockchain nodes. This adheres heavily to the Single Responsibility Principle, as each payment method has its own dedicated class. It also supports the Open/Closed Principle; if a new payment method is required, developers only need to create a new concrete strategy class and add a case to the controller, without modifying the core payment processing logic or existing strategies. The pattern also makes the payment algorithms interchangeable at runtime.

Disadvantages: The Strategy pattern increases the overall complexity of the program by introducing many new classes and interfaces. For a simple system with only a couple of payment methods that rarely change, this might be overengineering. Furthermore, the client (in this case, `PaymentController`) must be aware of the different strategies and understand the differences between them to select the right one. The communication overhead between the Context and the Strategy may also be a minor factor, as data might need to be passed back and forth, though our DTO (`PaymentRequest`) handles this gracefully here.

6. Conclusion

The Strategy pattern is highly effective in scenarios where an object needs to perform a specific task but can achieve it in several different ways, such as handling various payment methods in an e-commerce checkout system. In this program, `PaymentProcessor` handles the general context of processing a payment, while classes like `MidtransVAStrategy` and `StripeCreditCardStrategy` encapsulate the specific rules and API calls for each payment vendor.

This is clearly demonstrated in the `PaymentController`, where the appropriate strategy is instantiated and injected into the context based on the user's selection. By isolating the payment logic into distinct, interchangeable classes, the system becomes highly modular, extensible, and easier to test or modify. Overall, the Strategy pattern is a powerful behavioral design pattern for organizing algorithms into families, making them interchangeable, and hiding their complex implementation details from the client that uses them.